

Assignment 5

Problem Statement

To implement the **A* (A-Star) Search Algorithm** in Python using the heapq module to find the **optimal/shortest path** between a given start node and goal node in a weighted graph.

Algorithm :-

- Start with the initial node.
- Assign $g(n)$ and $h(n)$ values to each node.
- Calculate the total cost using:
$$f(n) = g(n) + h(n)$$
- Insert the starting node into a priority queue using heapq.
- Select the node with the **lowest $f(n)$** value.
- Check whether the selected node is the goal node.
- If it is the goal, display the path and total cost.
- Otherwise, expand its neighbouring nodes.
- Calculate the new $g(n)$, $h(n)$ and $f(n)$ values.
- Insert the unvisited nodes into the priority queue.
- Repeat the process until the goal node is reached.

Graph :-

Use this simple weighted graph:

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('A', 1), ('D', 2)],  
    'C': [('A', 4), ('F', 2)],  
    'D': [('B', 2), ('G', 3)],  
    'F': [('C', 2), ('G', 1)],  
    'G': [('D', 3), ('F', 1)]  
}
```

Coding :-

```

import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('D', 2)],
    'C': [('A', 4), ('F', 2)],
    'D': [('B', 2), ('G', 3)],
    'F': [('C', 2), ('G', 1)],
    'G': [('D', 3), ('F', 1)]
}

h = {
    'A': 5,
    'B': 4,
    'C': 2,
    'D': 2,
    'F': 1,
    'G': 0
}

def a_star(start, goal):
    queue = []
    # (f, g, node, path)
    heapq.heappush(queue, (h[start], 0, start, [start]))
    visited = set()
    while queue:
        f, g, node, path = heapq.heappop(queue)
        if node in visited:
            continue
        visited.add(node)
        print("Visiting:", node, "f =", f)

```

```
if node == goal:

    print("\nGoal Found!")

    print("Path:", " -> ".join(path))

    print("Cost:", g)

    return

for neighbour, cost in graph[node]:

    if neighbour not in visited:

        new_g = g + cost

        new_f = new_g + h[neighbour]

        heapq.heappush(

            queue,

            (new_f, new_g, neighbour, path + [neighbour])

        )

a_star('A', 'G')
```

Output :-

Visiting: A f = 5

Visiting: B f = 5

Visiting: C f = 6

Visiting: D f = 5

Visiting: F f = 7

Visiting: G f = 6

Goal Found!

Path: A -> B -> D -> G

Cost: 6

Conclusion:-

The **A* (A-Star) Search Algorithm** was successfully implemented in Python using the heapq module. The algorithm uses the evaluation function $f(n) = g(n) + h(n)$ to select the most promising node.

The optimal path from **A to G** was found as:

A → B → D → G

with a **total cost of 6**.